Quest 4: The Rankening

Lord of the Wings · scoring the Tournament of Elven Warriors

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. It explains what the quest asks and the logic behind my solutions, not the code itself. Every number in the examples and traces was checked by running my solutions on the sample data (plus small instrumented copies to print intermediate values).

THE QUESTION
Turn tournament records into a ranking, then compute a checksum: Σ position × (the value being ranked).

INPUT
One record per line, such as `id=7 quest=1 part=2 time=345`, in any order.

OUTPUT
The checksum.

WHAT CHANGES BETWEEN PARTS
What the ranking is based on (total time → parts completed then time → points with ties).

The story

The Grand Council agrees to help the journey to Nulldor, where the cursed Dragonduck egg Frodoff is carrying must be destroyed. The elves want to send their best warrior, but nobody has worked out the standings of the **Tournament of Elven Warriors** yet. You build the leaderboard.

Understanding the question

The records

Field	Meaning
`id`	the participant
`quest`	which tournament quest (one quest per day)
`part`	part 1, 2 or 3 of that quest
`time`	seconds taken for that part

Each record is a few `name=value` pairs, such as `id=1 quest=1 part=1 time=187`. Records are **in no particular order**, so you have to group them yourself, by participant and later by quest.

The checksum

```
checksum = 1 × (value of 1st place) + 2 × (value of 2nd place) + 3 × (value of 3rd place)
+ …
```

The "value" is total time in Parts I and II, and total points in Part III.

PART I One quest, rank by total time

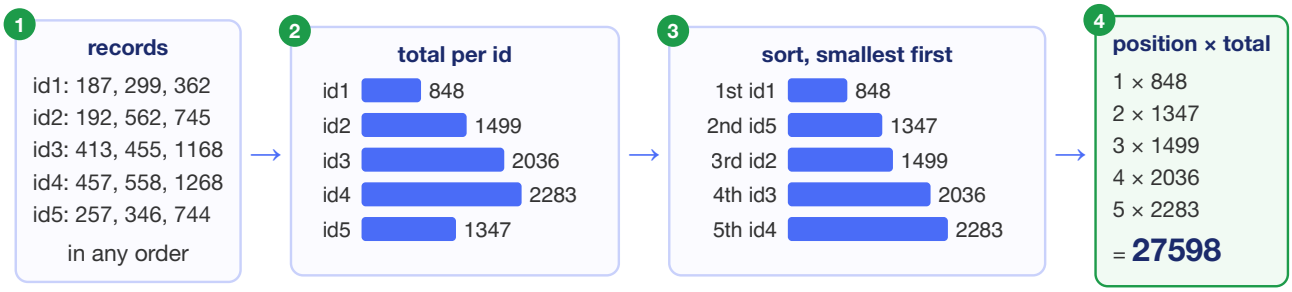
Every participant finished all three parts of one quest. Add each participant's three times and rank from **shortest total to longest**. There are no ties.

Step-by-step solution

1. Read each record: id, quest, part and time.
2. For each record, add its time to that participant's total.
3. Sort the participants by total time, smallest first.
4. Go down the list: position 1, 2, 3, …. Add `position × total time` for each.
5. That sum is the checksum.

How my solution works

Keep a **running total per participant** (record order doesn't matter), sort the totals, and weight each by its position:



Dots match the step numbers above.

PART II Whole tournament, more parts completed comes first

Now the records cover all 20 quests, and most participants didn't finish everything. The ranking rule is:

1. **More completed parts always ranks higher**, whatever the time.
2. With the same number of parts, the **shorter total time** ranks higher.

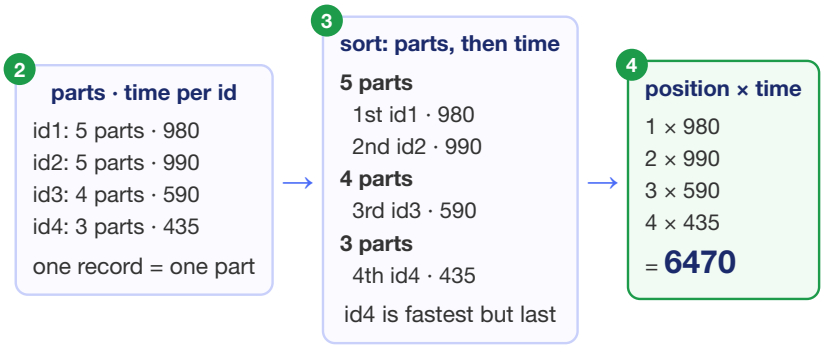
The checksum still uses total time. There are no ties.

Step-by-step solution

1. Read each record.
2. For each participant, keep two running numbers: total time, and how many records (parts) they have.
3. Sort by parts, most first. If two participants have the same number of parts, put the smaller total time first.
4. Add `position × total time` down the list.

How my solution works

Each participant now has **two** running numbers: parts completed and total time. Sort by parts first, then time:



Dots match the step numbers above.

PART III Ranking points, with ties

The final ranking is now based on **points**. For every quest, a separate mini-ranking is made after **each part**:

1. **Who is included:** everyone who completed that part. Anyone who didn't gets 0 points for it.
2. **What is compared:** the **combined** time so far. After part 2 that is part 1 + part 2; after part 3 it is all three.
3. **Points:** first place gets **50** (part 1), **100** (part 2) or **150** (part 3). Each lower position gets 1 point less, down to 1, and anyone below that gets 0. So `points = max(50 × part − (position − 1), 0)`.
4. **Ties:** equal combined times **share a position** and each tied participant gets that position's points. The next participant's position **skips ahead**: 1, 1, 3, not 1, 1, 2.

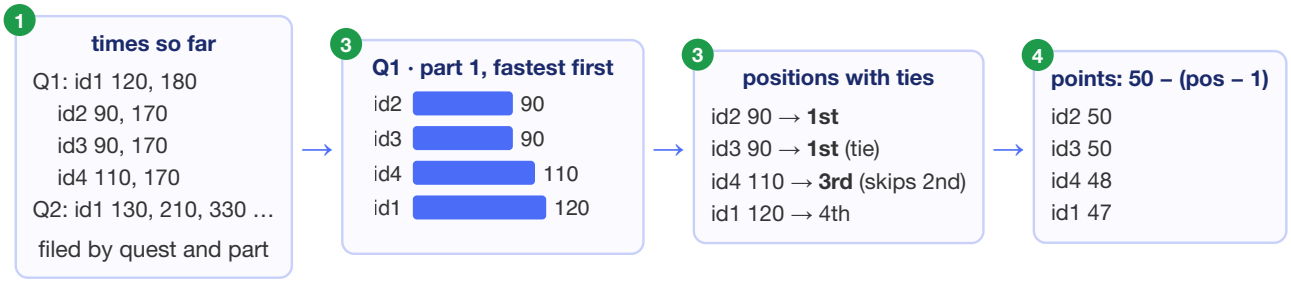
Add up each participant's points, rank by total points (highest first), and compute `Σ position × points`. **Tied totals share a position here as well:** `… + 55×374 + 55×374 + 57×…`.

Step-by-step solution

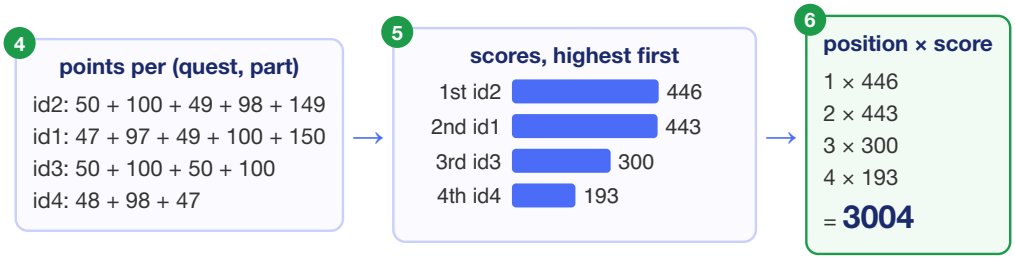
1. Read each record and file its time under its quest, its participant and its part.
2. For each quest, and each part k = 1, 2, 3: collect everyone who completed part k, with their **combined** time for parts 1…k.
3. Sort that list, fastest first. Give out positions, where equal times share a position and the next one skips ahead (1, 1, 3).
4. Each participant gets `max(50·k − (position − 1), 0)` points, added to their total score.
5. When every quest and part is done, sort the participants by total score, highest first. Give positions with the same tie rule.
6. Add `position × score` down the list. That is the checksum.

How my solution works

File every time by quest, participant and part. Then, for every (quest, part), rank everyone who finished it by their **combined** time so far and hand out points. Example (sample3), quest 1 part 1:



Doing that for every (quest, part) and adding up gives each participant's score. A part nobody finished (Q1 · 3) is skipped. Then rank the scores, with the same tie rule:



Dots match the step numbers above. On the Part II data (no ties) the same solution gives the puzzle's 2998.

Easy-to-miss details in Part III, all handled by the solution:

- Mini-rankings are made separately for each **(quest, part)**, not across the whole tournament.
- Rankings after part 2 and part 3 use **running totals**, not the time for that part alone.
- Ties use **1, 1, 3** ranking, both when giving out points and in the final checksum.
- Positions far down still get 0 points rather than negative points.

Summary

Part	Ranked by	Ties?	Checksum multiplies
I	total time (low first)	no	total time
II	parts completed (high first), then total time (low first)	no	total time
III	total points from per-(quest, part) mini-rankings	yes, 1-1-3 style	total points